PS 1

Muchang Bahng

Fall 2026

Exercise 1

In the 2020 US Presidential election, there was a significant gap between the probability of a Biden victory assigned by poll aggregators (e.g., fivethirtyeight.com) and betting markets. According to Bayesian decision theory, a rational agent should choose actions (in this case, bet size) by writing down:

- a probability distribution $P(X)$ over the state X of the world describing what they think will happen,
- a utility function U describing the value of each action under a certain state of the world,

and then choose the action that maximizes their expected utility (equivalently, minimizes expected risk). That is, a rational agent will choose

$$a^* = \operatorname*{argmax}_a \mathbb{E}_{X\sim P}\left[U(a,X)\right].$$

Suppose that Victor is a rational agent with d dollars of total wealth, and that he thinks $P(\text{Biden wins}) = 0.85$ (the fivethirtyeight prediction), and that betting markets give even money bets at $P(\text{Biden wins}) = 0.6$.

(a) Prove that if Victor's utility function is just the amount of money he has then he'll bet d (his entire bankroll) on Biden winning. Does this kind of linear utility of money make sense?

(b) Now, suppose that Victor's utility from y dollars is simply $U(y) = \log(y)$. How much (of his d dollars) should he bet on Biden winning?

(c) Suppose you had a net worth of \$500,000. Would you personally make the bet from part (b)?

Solution. Throughout, a bet of size $x \in [0,d]$ on Biden at market probability $q = 0.6$ pays a profit of bx with $b = \frac{1-q}{q} = \frac{2}{3}$ if Biden wins, and loses x otherwise. Let $p = 0.85$ be Victor's belief.

(a) With $U(y) = y$, the final wealth is $d + bx$ with probability p and $d - x$ with probability $1 - p$, so

$$\mathbb{E}[U(x,X)] = p(d+bx) + (1-p)(d-x) = d + x\big(pb - (1-p)\big) = d + x\left(0.85\cdot\tfrac{2}{3} - 0.15\right) = d + \tfrac{5}{12}x.$$

This is strictly increasing in x, so the maximizer over $x \in [0,d]$ is $x^* = d$: he bets his entire bankroll. In general, $x^* = d$ whenever $pb > 1 - p$, i.e. $p > q$.

Linear utility of money does not make sense: it says that Victor is indifferent between \$1 for sure and a 50% chance of \$2, and would risk ruin (a 15% chance of losing everything) for an expected gain of $\frac{5}{12}d$. It ignores the diminishing marginal value of money and the fact that losing all wealth is catastrophic.

(b) Now maximize

$$g(x) = p\log(d+bx) + (1-p)\log(d-x), \qquad x \in [0,d).$$

g is strictly concave, and $g(x) \to -\infty$ as $x \to d$, so the maximizer is interior and is given by $g'(x) = 0$:

$$\frac{pb}{d+bx} = \frac{1-p}{d-x} \implies pb(d-x) = (1-p)(d+bx) \implies xb = d\big(pb-(1-p)\big) \implies x^* = d\left(p - \frac{1-p}{b}\right).$$

(This is the Kelly criterion.) Plugging in, $x^* = d\left(0.85 - \frac{0.15}{2/3}\right) = d(0.85 - 0.225) = 0.625\,d$. So Victor should bet 62.5% of his wealth.

(c) With $d =$ \$500,000, the Kelly bet is \$312,500. Personally I would not make this bet. The calculation takes the 0.85 forecast as exactly correct, but if the forecast is overconfident (the market is saying 0.6, and the market has money on the line), the optimal bet shrinks quickly; e.g. with $p = 0.6$ the optimal bet is 0. Kelly is also known to be aggressive: it is very sensitive to errors in p and to model misspecification. Furthermore, a 15% chance of losing \$312,500 (over half of my net worth) is a risk I would not take in a single event, and my true utility over this range is more risk averse than log. A fractional Kelly bet would be more reasonable.

Exercise 2

Suppose that Y is a categorical random variable, and X are covariates that we wish to use to predict Y. A loss function L on categorical data is called a *proper scoring rule* if the predictive function f^* that minimizes the implied risk is the conditional probability of Y given X. That is, if

$$f^* \coloneqq \operatorname*{argmin}_f \mathbb{E}_{X,Y}\left[L\left(Y, f(X)\right)\right] \text{ is given by } f^*(x) = P(Y \mid X = x).$$

Prove that the Cross Entropy loss is a proper scoring rule in the case where Y is binary.

Solution. Let $Y \in \{0, 1\}$ and let $f(x) \in (0, 1)$ denote the predicted probability that $Y = 1$. The cross entropy loss is $L(y, f(x)) = -\big[y \log f(x) + (1 - y)\log(1 - f(x))\big]$. By the tower property,

$$\mathbb{E}_{X,Y}[L(Y, f(X))] = \mathbb{E}_X\Big[\mathbb{E}_{Y|X}\big[L(Y, f(X))\big]\Big],$$

so the risk is minimized by minimizing the inner conditional expectation separately at each x (we can choose the value $f(x)$ freely for each x). Fix x and let $p = P(Y = 1 \mid X = x)$ and $q = f(x)$. Then

$$\phi(q) \coloneqq \mathbb{E}[L(Y, q) \mid X = x] = -p \log q - (1 - p)\log(1 - q).$$

Differentiating,

$$\phi'(q) = -\frac{p}{q} + \frac{1 - p}{1 - q} = \frac{q - p}{q(1 - q)},$$

which is negative for $q < p$ and positive for $q > p$ (when $p \in (0, 1)$). Hence ϕ is strictly decreasing and then strictly increasing, with unique minimizer $q = p$. (Equivalently, $\phi(q) = H(p) + \mathrm{KL}(\mathrm{Ber}(p) \,\|\, \mathrm{Ber}(q)) \geq H(p)$, with equality iff $q = p$.) If $p = 0$ or 1, $\phi(q) = -\log(1 - q)$ or $-\log q$, which is minimized (as an infimum) at $q = p$ as well.
Therefore the pointwise minimizer is $f^*(x) = P(Y = 1 \mid X = x)$ for every x, so $f^* = \operatorname{argmin}_f \mathbb{E}[L(Y, f(X))]$ is the conditional probability, and cross entropy is a proper scoring rule. ■

Exercise 3

As we saw in class, a requirement for stochastic gradient descent to converge to the minimum of an objective function is that the step size ρ_t decays fast enough that $\sum_t \rho_t^2 < \infty$. In actual machine learning practice, it's a common practice to just use a constant step size, violating the convergence requirement. Why might this be reasonable in a finite sample setting?

Solution. The condition $\sum_t \rho_t^2 < \infty$ (together with $\sum_t \rho_t = \infty$) is what kills the gradient noise so that SGD converges exactly to the minimizer of the objective. In practice this is not what we want, for several reasons.

1. **Finite compute.** We only run a finite number T of steps, so $\sum_{t \le T} \rho^2 = T\rho^2 < \infty$ trivially; the asymptotic condition is never actually tested. Meanwhile, a decaying schedule like $\rho_t \sim 1/t$ shrinks so quickly that progress is very slow in the time budget we actually have, whereas a constant step size makes (for strongly convex smooth objectives) geometric progress toward the minimizer until it reaches a noise floor.
2. **The noise floor is of the same order as the statistical error.** With constant step size ρ, SGD converges linearly to a neighborhood of the minimizer whose radius is $O(\rho\sigma^2)$ in squared distance, where σ^2 is the gradient noise variance. Meanwhile, the empirical risk minimizer computed from n samples is itself only within $O(1/\sqrt{n})$ of the true population minimizer, so minimizing the empirical risk to higher precision than this does not improve the test performance. It suffices to choose ρ so that the optimization error is below the statistical error.
3. **Implicit regularization / generalization.** Using a constant (or large) step size and stopping early keeps the iterates from fitting noise in the training set, and the noise in SGD with a constant step size biases the iterates toward flatter minima that tend to generalize better. Converging exactly to the empirical minimizer can overfit, especially for overparameterized models.
4. **Non-convexity and non-stationarity.** In non-convex problems the goal is to reach a good region rather than a particular minimizer, and a non-vanishing step size helps the iterates escape saddle points and poor local minima. If the data distribution drifts, a constant step size also lets the model keep tracking it, whereas a decaying step size would freeze.

In short, constant step size gives fast progress to within the noise level, and that noise level is already below the precision the finite sample can justify. (In practice one often uses a constant step size and then decays it at the end, or uses a schedule, to reduce the final noise floor.)

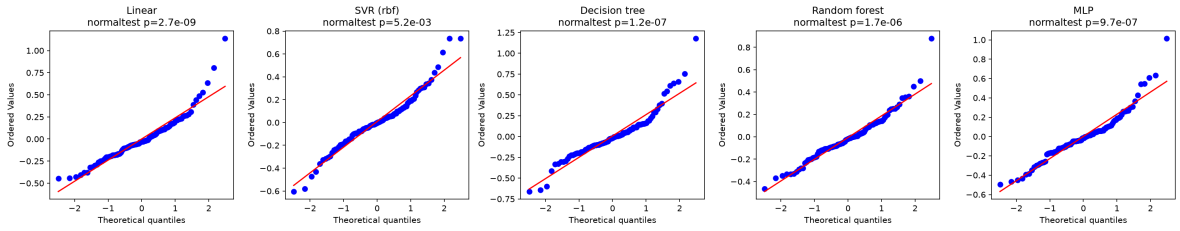
Exercise 4

One common justification for using square error loss in machine learning is that the residuals of the fitted model should be (approximately) normal. This justification can be approximately tested by fitting a model to the data and then testing the residuals for normality! To do that, you are asked to fit several ML models to the Average Localization Error (ALE) dataset and run a statistical test to see if the residuals are approximately normally distributed. You should predict `ale` using the first 4 variables (you can ignore `sd_ale`). You can find different regression models in the `sklearn` package. Try whatever you like (for example, linear model, svm, decision tree, or neural network). To test the normality, you are allowed to use any test method. One recommendation is to draw a `qqplot` and run `normaltest` in the `scipy` package.

Solution. I fit five models to predict `ale` from `anchor_ratio`, `trans_range`, `node_density` and `iterations`: linear regression, an RBF-kernel SVR, a depth-5 decision tree, a random forest, and a two-hidden-layer MLP (the scale-sensitive models were preceded by standardization). To keep overfitting from shrinking the residuals, I used out-of-fold predictions from 10-fold cross validation, with residuals $e_i = y_i - \hat{y}_i$. I tested

$$H_0 : \text{the residuals are normally distributed} \quad \text{vs.} \quad H_1 : \text{they are not}$$

with `scipy.stats.normaltest` (the D'Agostino–Pearson test, which combines skewness and kurtosis) and drew a Q–Q plot for each model.



Model	Std of residual	Skewness	Excess kurtosis	`normaltest` p
Linear	0.246	+1.34	+4.03	2.7×10^{-9}
SVR (rbf)	0.226	+0.43	+1.71	5.2×10^{-3}
Decision tree	0.265	+1.07	+3.71	1.2×10^{-7}
Random forest	0.196	+0.94	+3.08	1.7×10^{-6}
MLP	0.234	+1.00	+3.03	9.7×10^{-7}

Every model has $p < 0.01$, so at $\alpha = 0.05$ we reject normality for all five. The Q–Q plots agree: the points bend away from the line in the upper tail, and the residuals are right-skewed and heavy-tailed (positive skewness and excess kurtosis). Even the best-behaved model (SVR) is rejected. So the normality justification for squared error loss does not hold well on this dataset, which motivates the heavier-tailed t loss in the regression part of Problem 5.

```
import numpy as np, pandas as pd, torch
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy import stats
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_predict
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.svm import SVR
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import accuracy_score, mean_absolute_error

SEED = 0
torch.manual_seed(SEED); np.random.seed(SEED)
torch.set_default_dtype(torch.float64)

ale = pd.read_csv("data/ale.csv")
X_ale = ale[["anchor_ratio", "trans_range", "node_density", "iterations"]].values
y_ale = ale["ale"].values

# ------------------------------------------------------------------ Problem 4
print("=" * 70, "\nProblem 4: normality of residuals\n" + "=" * 70)
models = {
    "Linear": make_pipeline(StandardScaler(), LinearRegression()),
    "SVR (rbf)": make_pipeline(StandardScaler(), SVR(C=10, epsilon=0.01)),
    "Decision tree": DecisionTreeRegressor(max_depth=5, random_state=SEED),
    "Random forest": RandomForestRegressor(n_estimators=200, random_state=SEED),
    "MLP": make_pipeline(StandardScaler(), MLPRegressor(hidden_layer_sizes=(64, 64),
    max_iter=5000, random_state=SEED)),
}
fig, axes = plt.subplots(1, len(models), figsize=(4 * len(models), 4))
for ax, (name, m) in zip(axes, models.items()):
    # out-of-fold predictions so residuals are not distorted by overfitting
    resid = y_ale - cross_val_predict(m, X_ale, y_ale, cv=10)
    k2, p = stats.normaltest(resid)
    print(f"{name:14s} resid mean={resid.mean():+.4f} std={resid.std():.4f} "
          f"skew={stats.skew(resid):+.2f} kurt={stats.kurtosis(resid):+.2f} "
          f"normaltest stat={k2:.2f} p={p:.2e}")
    stats.probplot(resid, dist="norm", plot=ax)
    ax.set_title(f"{name}\nnormaltest p={p:.1e}")
plt.tight_layout(); plt.savefig("figs/qqplots.png", dpi=150); plt.close()
```

Exercise 5 (Classification)

For this part, you will use the Breast Cancer Wisconsin dataset, which you can load using `sklearn.datasets.load_breast_cancer`. The dataset contains 30 features and a binary label. Begin by splitting the data (80/20) into a training and test set.
As a baseline, use sklearn's implementation to train a logistic regression model on the training data and evaluate its accuracy on the test set, where accuracy is defined as the proportion of correctly classified observations.

(a) Now you will implement a logistic regression model using a framework that supports automatic differentiation. Begin by defining a logistic regression model:

$$f(x_i) = \sigma(w^T x_i + b)$$

where x_i are the features for observation i, w are the weights, b is the bias, and σ is the sigmoid function. Next, define a function for the binary cross-entropy loss:

$$L(y_i, f(x_i)) = -\left[y_i \log(f(x_i)) + (1 - y_i) \log(1 - f(x_i))\right]$$

Use your chosen framework to implement this model and train it on the training data by minimizing the loss using SGD. Note that you may need to clip the output of the sigmoid function to avoid numerical instability. Evaluate the performance of your model on the test set. How does it compare to the baseline?

(b) Logistic regression uses the function $\sigma(z_i)$ to map the real-valued $z_i := w^T x_i + b$ to a probability between 0 and 1. However, this is not the only way to map a real-valued output to a probability. Probit regression uses $\Phi(z_i)$, where Φ is the cumulative distribution function of the standard normal distribution. The model is then:

$$f(x_i) = \Phi(w^T x_i + b)$$

Implement a probit regression model. Using the same binary cross-entropy loss as before, train the model on the training data by minimizing the loss with SGD. As before, you may need to clip the output of the cumulative distribution function to avoid numerical instability. Evaluate the performance of your model on the test set. How does it compare to the baseline?

Solution. I used PyTorch (`torch.optim.SGD`, minibatches of 32, learning rate 0.1, 200 epochs, zero initialization, double precision) on an 80/20 stratified split (`random_state`= 0, 114 test points) with standardized features. The model outputs were clipped to $[10^{-7}, 1 - 10^{-7}]$ inside the loss, and probit used `torch.distributions.Normal(0, 1).cdf`. The full code is in `hw1_code.py`.

Model	Test accuracy
sklearn `LogisticRegression` (baseline)	0.9825
(a) Logistic regression, BCE + SGD	0.9825
(b) Probit regression, BCE + SGD	0.9825

(a) The logistic regression trained by SGD matches the baseline at 98.25% (112 of 114 correct).

(b) The probit regression also matches the baseline at 98.25%. Since $\Phi(z) \approx \sigma(1.7z)$, the two link functions differ mainly by a rescaling of w and b, so it is unsurprising that they classify equally well here (the classification threshold $f = 0.5$ is the hyperplane $w^T x + b = 0$ in both cases).

On this test set the three models are indistinguishable by accuracy.

```
# ------------------------------------------------------------- helpers
def sgd_fit(loss_fn, params, Xtr, ytr, lr, epochs, batch=32, seed=SEED):
```

```
    """Minibatch SGD with torch.optim.SGD; loss_fn(params, X, y) -> scalar."""
    g = torch.Generator().manual_seed(seed)
    opt = torch.optim.SGD(params, lr=lr)
    n = len(Xtr)
    for _ in range(epochs):
        perm = torch.randperm(n, generator=g)
        for i in range(0, n, batch):
            idx = perm[i:i + batch]
            opt.zero_grad()
            loss_fn(params, Xtr[idx], ytr[idx]).backward()
            opt.step()
    return params

def to_t(*arrs):
    return [torch.tensor(a, dtype=torch.float64) for a in arrs]

# ------------------------------------------------------------------ Problem 5: classification
print("\n" + "=" * 70, "\nProblem 5: classification (Breast Cancer Wisconsin)\n" + "=" * 70)
Xc, yc = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(Xc, yc, test_size=0.2, random_state=SEED, stratify=yc)
sc = StandardScaler().fit(Xtr)
Xtr_s, Xte_s = sc.transform(Xtr), sc.transform(Xte)

base = LogisticRegression(max_iter=5000).fit(Xtr_s, ytr)
print(f"Baseline sklearn LogisticRegression test acc: {accuracy_score(yte,
    base.predict(Xte_s)):.4f}")

Xtr_t, Xte_t, ytr_t = to_t(Xtr_s, Xte_s, ytr.astype(float))
EPS = 1e-7
std_normal = torch.distributions.Normal(0.0, 1.0)

def make_bce(link):
    def loss(params, X, y):
        w, b = params
        f = link(X @ w + b).clamp(EPS, 1 - EPS)          # clip for stability
        return -(y * torch.log(f) + (1 - y) * torch.log(1 - f)).mean()
    return loss

for name, link in [("(a) Logistic (sigmoid)", torch.sigmoid), ("(b) Probit (Normal CDF)",
    std_normal.cdf)]:
    w = torch.zeros(Xtr_t.shape[1], requires_grad=True)
    b = torch.zeros(1, requires_grad=True)
    sgd_fit(make_bce(link), [w, b], Xtr_t, ytr_t, lr=0.1, epochs=200)
    with torch.no_grad():
        pred = (link(Xte_t @ w + b) > 0.5).numpy().astype(int)
    print(f"{name:25s} test acc: {accuracy_score(yte, pred):.4f}")
```

Exercise 5 (Regression)

For this part, you will use the average localization error (ALE) dataset from the previous question. Begin by splitting the data (80/20) into a training and test set.

As a baseline, use sklearn's implementation to train a linear regression model on the training data and evaluate its performance on the test set using the mean absolute error.

(a) Now you will implement a linear regression model using a framework that supports automatic

differentiation. Begin by defining a linear regression model:

$$f(x_i) = w^T x_i + b$$

where x_i are the features for observation i, w are the weights, and b is the bias. Next, define a function for the mean squared error loss:

$$L(y_i, f(x_i)) = (y_i - f(x_i))^2$$

Use your chosen framework to implement this model and train it on the training data by minimizing the loss with SGD. Evaluate the performance of your model on the test set. How does it compare to the baseline?

(b) Earlier you saw that the squared error loss can be derived from the assumption that the residuals of the fitted model should be (approximately) normal. However, the normal distribution has weak tails, so it is sensitive to outliers. An alternative is the t-distribution, which has heavier tails. The t-distribution is parameterized by the degrees of freedom ν. When $\nu \to \infty$, the t-distribution converges to the standard normal distribution. The probability density function of the t-distribution is:

$$p(y_i - f(x_i), \nu) = \frac{\Gamma\left(\frac{\nu+1}{2}\right)}{\sqrt{\nu\pi}\,\Gamma\left(\frac{\nu}{2}\right)} \left(1 + \frac{(y_i - f(x_i))^2}{\nu}\right)^{-\frac{\nu+1}{2}}$$

where Γ is the gamma function. The corresponding loss function is the negative log-likelihood:

$$L(y_i, f(x_i)) = -\log p(y_i - f(x_i), \nu)$$

For this exercise, use the t-distribution with $\nu = 5$. If you are using JAX, you can use `jax.scipy.stats.t.logpdf` to implement this loss function. If you are using PyTorch, you can use `torch.distributions.StudentT.log_prob`.

Use your chosen framework to implement a linear regression model and train it on the training data by minimizing this custom loss function with SGD. Evaluate the performance of your model on the test set. How does it compare to the baseline?

Solution. I used the same setup as before: PyTorch with `torch.optim.SGD` (minibatches of 32, learning rate 0.05, 300 epochs, $w = 0$, b initialized to the mean of the training targets), an 80/20 split (`random_state`= 0, 22 test points) and standardized features. For (b) I wrote the loss as the explicit negative log-likelihood,

$$-\log p(r, \nu) = -\log\Gamma\left(\tfrac{\nu+1}{2}\right) + \log\Gamma\left(\tfrac{\nu}{2}\right) + \tfrac{1}{2}\log(\nu\pi) + \tfrac{\nu+1}{2}\log\left(1 + \tfrac{r^2}{\nu}\right), \qquad r = y - f(x),$$

with $\nu = 5$ (only the last term depends on the parameters).

Model	w_{anchor}	w_{range}	w_{density}	w_{iter}	b	Test MAE
sklearn baseline	−0.054	−0.072	−0.293	−0.214	1.018	0.1921
(a) Squared error SGD	−0.052	−0.073	−0.291	−0.213	1.019	0.1931
(b) Student-t NLL + SGD	−0.053	−0.070	−0.290	−0.209	1.016	0.1898

(Weights are on standardized features, in the order `anchor_ratio`, `trans_range`, `node_density`, `iterations`.)

(a) SGD on the squared error reaches a test MAE of 0.1931 versus 0.1921 for the baseline. This is expected, since both minimize the same convex objective and SGD converges to (a neighborhood of) the least squares solution; the fitted coefficients agree with the closed-form ones to about two decimals.

(b) SGD on the Student-t loss reaches a test MAE of 0.1898, which is about 1% lower than the baseline. This is in the direction one would expect from the heavy-tailed residuals found in Problem 4, since the t loss grows only logarithmically in the residual and so downweights outliers.

However, the difference is tiny, and with only 22 test points it is well within noise, so I would conclude that the model performs about the same as the baseline.

```
# ------------------------------------------------------------------ Problem 5: regression
print("\n" + "=" * 70, "\nProblem 5: regression (ALE)\n" + "=" * 70)
Xtr, Xte, ytr, yte = train_test_split(X_ale, y_ale, test_size=0.2, random_state=SEED)
sc = StandardScaler().fit(Xtr)
Xtr_s, Xte_s = sc.transform(Xtr), sc.transform(Xte)

base = LinearRegression().fit(Xtr_s, ytr)
print(f"Baseline sklearn LinearRegression test MAE: {mean_absolute_error(yte,
    base.predict(Xte_s)):.4f}")

Xtr_t, Xte_t, ytr_t = to_t(Xtr_s, Xte_s, ytr)
NU = 5.0

def mse_loss(params, X, y):
    w, b = params
    return ((y - (X @ w + b)) ** 2).mean()

def t_nll(params, X, y):
    w, b = params
    r = y - (X @ w + b)
    # -log p(r; nu) = -lgamma((nu+1)/2) + lgamma(nu/2) + 0.5 log(nu pi) + (nu+1)/2 log(1 +
    r^2/nu)
    return (-torch.lgamma(torch.tensor((NU + 1) / 2)) + torch.lgamma(torch.tensor(NU / 2))
            + 0.5 * np.log(NU * np.pi) + (NU + 1) / 2 * torch.log1p(r ** 2 / NU)).mean()

for name, loss in [("(a) Squared error + SGD", mse_loss), (f"(b) Student-t (nu={NU:g}) NLL +
    SGD", t_nll)]:
    w = torch.zeros(Xtr_t.shape[1], requires_grad=True)
    b = torch.tensor([float(ytr.mean())], requires_grad=True)   # start at mean of y
    sgd_fit(loss, [w, b], Xtr_t, ytr_t, lr=0.05, epochs=300)
    with torch.no_grad():
        mae = (Xte_t @ w + b - torch.tensor(yte)).abs().mean().item()
    print(f"{name:36s} test MAE: {mae:.4f}   w={w.detach().numpy().round(3)}
    b={b.item():.3f}")
```